SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	**K.Vijay Bhaskar Reddy**	**Reg.No.:**	**192425155**
Department:	**AIML**	**Date:**	**13/08/2026**

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I K.Vijay Bhaskar Reddy(192425155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1 — Register Transfer and ALU Operations

Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Aim

To design and simulate a register transfer system with multiple registers and an ALU to perform **addition, subtraction, AND, and OR operations**, and to observe the transfer of data between registers.

Procedure

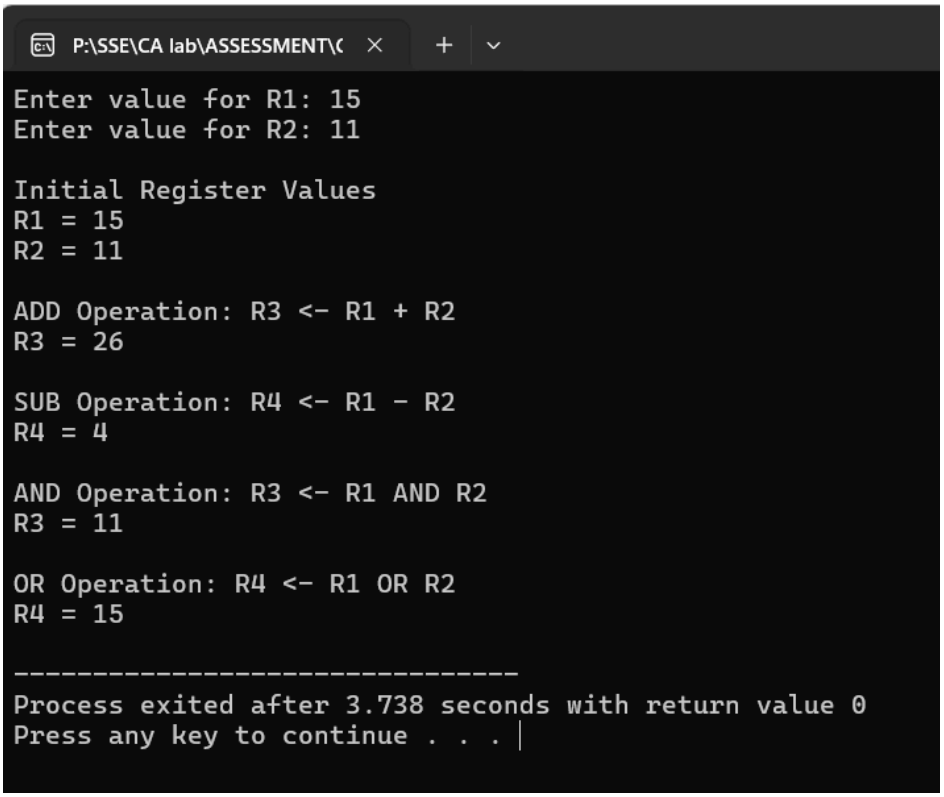
1. Create multiple registers such as **R1, R2, R3, and R4**.
2. Provide input data to R1 and R2.
3. Connect the registers to the ALU through a common bus.
4. Configure control signals to select the required source registers.
5. Transfer the contents of R1 and R2 to the ALU.
6. Select the required ALU operation:
    - ADD
    - SUBTRACT
    - AND
    - OR
7. Store the ALU output in R3.
8. Transfer the result from R3 to R4.
9. Observe the register contents after every clock cycle.
10. Verify the output with the expected arithmetic and logical results.

Program / Simulation

```
#include <stdio.h>
int main()
{
   int R1, R2, R3, R4;
   printf("Enter value for R1: ");
   scanf("%d", &R1);
   printf("Enter value for R2: ");
   scanf("%d", &R2);
   printf("\nInitial Register Values\n");
   printf("R1 = %d\n", R1);
   printf("R2 = %d\n", R2);
   R3 = R1 + R2;
   printf("\nADD Operation: R3 <- R1 + R2\n");
   printf("R3 = %d\n", R3);
   R4 = R1 - R2;
   printf("\nSUB Operation: R4 <- R1 - R2\n");
```

```
    printf("R4 = %d\n", R4);
    R3 = R1 & R2;
    printf("\nAND Operation: R3 <- R1 AND R2\n");
    printf("R3 = %d\n", R3);
    R4 = R1 | R2;
    printf("\nOR Operation: R4 <- R1 OR R2\n");
    printf("R4 = %d\n", R4);
    return 0;
}
```

Output



Result

The register transfer system was successfully simulated. The ALU correctly performed **addition, subtraction, AND, and OR operations**, and the results were successfully stored in the destination registers.

Conclusion

The experiment demonstrates how **registers, buses, control signals, and the ALU** work together during register transfer operations. The ALU performs the required arithmetic and logical processing while registers store and transfer the data.

Experiment 2 — Single-Bus and Multi-Bus Organization

Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches.

Aim

To design and simulate **single-bus and multi-bus computer organizations**, perform data transfers between registers, memory, and I/O, and compare their performance and efficiency.

Procedure

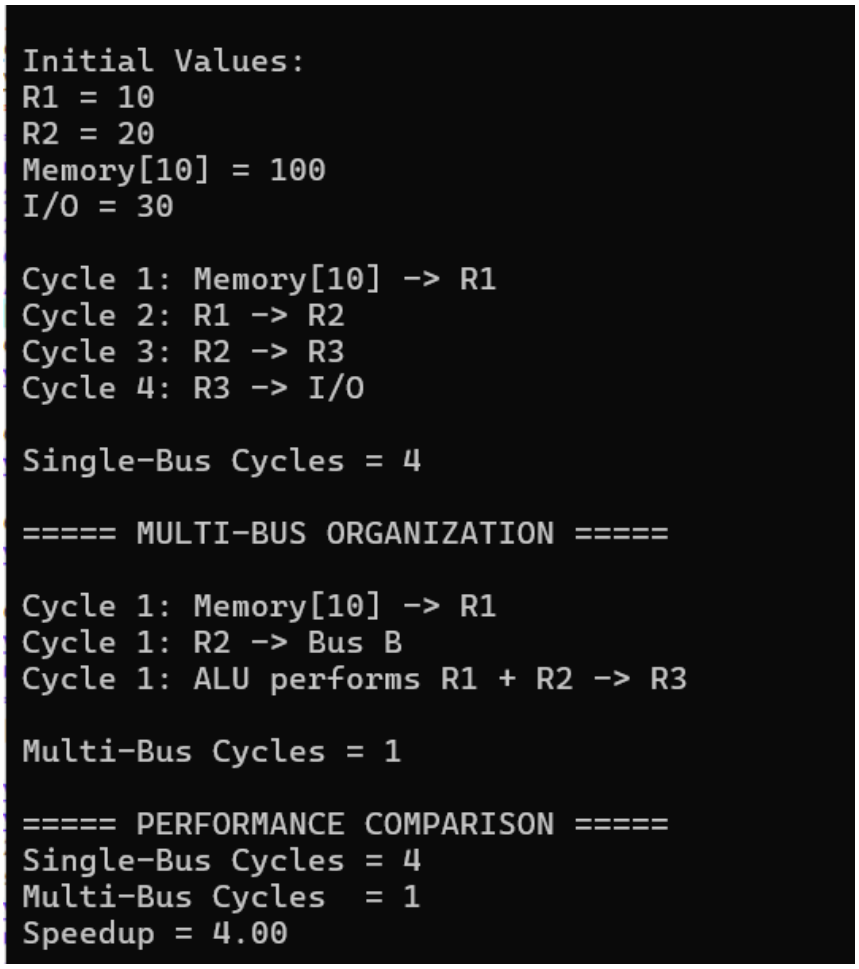
1. Create registers such as R1, R2, and R3.
2. Create a memory unit and I/O unit.
3. First connect all components using a **single common bus**.
4. Perform register-to-register data transfers.
5. Perform memory read and write operations.
6. Perform I/O data transfers.
7. Record the number of clock cycles required.
8. Modify the design to use a **multi-bus structure**.
9. Allow multiple data transfers to occur simultaneously.
10. Record the clock cycles again.
11. Compare the execution time and efficiency of both architectures.

Program / Simulation

```
#include <stdio.h>
int main()
{
   int R1 = 10, R2 = 20, R3 = 0;
   int memory[50];
   int io = 30;
   int single_cycles = 0;
   int multi_cycles = 0;
   memory[10] = 100;
   printf("===== SINGLE-BUS ORGANIZATION =====\n");
   printf("\nInitial Values:");
   printf("\nR1 = %d", R1);
   printf("\nR2 = %d", R2);
   printf("\nMemory[10] = %d", memory[10]);
   printf("\nI/O = %d\n", io);
   R1 = memory[10];
   single_cycles++;
   printf("\nCycle %d: Memory[10] -> R1", single_cycles);
   R2 = R1;
   single_cycles++;
   printf("\nCycle %d: R1 -> R2", single_cycles);
   R3 = R2;
   single_cycles++;
```

```
    printf("\nCycle %d: R2 -> R3", single_cycles);
    io = R3;
    single_cycles++;
    printf("\nCycle %d: R3 -> I/O", single_cycles);
    printf("\n\nSingle-Bus Cycles = %d\n", single_cycles);
    printf("\n===== MULTI-BUS ORGANIZATION =====\n");
    R1 = memory[10];
    R2 = 20;
    printf("\nCycle 1: Memory[10] -> R1");
    printf("\nCycle 1: R2 -> Bus B");
    R3 = R1 + R2;
    multi_cycles++;
    printf("\nCycle %d: ALU performs R1 + R2 -> R3", multi_cycles);
    printf("\n\nMulti-Bus Cycles = %d\n", multi_cycles);
    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("Single-Bus Cycles = %d\n", single_cycles);
    printf("Multi-Bus Cycles  = %d\n", multi_cycles);
    printf("Speedup = %.2f\n",
           (float)single_cycles / multi_cycles);
    return 0;
}
```

Output



Result

The single-bus and multi-bus organizations were successfully simulated. The multi-bus architecture required fewer clock cycles for operations involving multiple simultaneous transfers.

Conclusion

The experiment demonstrates that a **single-bus architecture is simpler but creates a communication bottleneck**, whereas a **multi-bus architecture allows parallel data transfers**, improving execution speed and overall system efficiency.

Experiment 3 — Five-Stage Instruction Pipeline

Simulate a 5-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory, and Write-back stages. Execute a set of instructions and compare its performance with a non-pipelined system.

Aim

To simulate a **5-stage instruction pipeline** and calculate its speedup and throughput compared with a non-pipelined processor.

Procedure

1. Divide instruction execution into five stages:
    - **IF – Instruction Fetch**
    - **ID – Instruction Decode**
    - **EX – Execute**
    - **MEM – Memory Access**
    - **WB – Write Back**
2. Select a set of instructions.
3. Execute the instructions using the non-pipelined approach.
4. Record the total number of clock cycles.
5. Execute the same instructions using the five-stage pipeline.
6. Allow different instructions to occupy different stages simultaneously.
7. Record the pipeline execution time.
8. Calculate speedup.
9. Calculate instruction throughput.
10. Compare both systems.

Program / Simulation

```
#include <stdio.h>
#define STAGES 5
#define INSTRUCTIONS 5
int main()
{
    char *stage[] = {
        "IF", "ID", "EX", "MEM", "WB"
    };
    int i, cycle;
    int pipeline_cycles = STAGES + INSTRUCTIONS - 1;
    int non_pipeline_cycles = STAGES * INSTRUCTIONS;
    printf("===== 5-STAGE PIPELINE =====\n\n");
    printf("Stages: IF -> ID -> EX -> MEM -> WB\n\n");
    for (cycle = 1; cycle <= pipeline_cycles; cycle++)
    {
        printf("Cycle %d: ", cycle);
        for (i = 0; i < INSTRUCTIONS; i++)
        {
```

```
            int stage_no = cycle - i - 1;

            if (stage_no >= 0 && stage_no < STAGES)
            {
                printf("I%d-%s  ",
                    i + 1,
                    stage[stage_no]);
            }
        }
        printf("\n");
    }
    printf("\n===== PERFORMANCE =====\n");
    printf("Number of Instructions = %d\n", INSTRUCTIONS);
    printf("Pipeline Stages = %d\n", STAGES);
    printf("Non-Pipelined Cycles = %d\n",
           non_pipeline_cycles);
    printf("Pipelined Cycles = %d\n",
           pipeline_cycles);
    printf("Speedup = %.2f\n",
           (float)non_pipeline_cycles /
           pipeline_cycles);
    printf("Throughput = %.2f instructions/cycle\n",
           (float)INSTRUCTIONS /
           pipeline_cycles);
    return 0;
}
```

Output:

```
===== 5-STAGE PIPELINE =====

Stages: IF -> ID -> EX -> MEM -> WB

Cycle 1: I1-IF
Cycle 2: I1-ID  I2-IF
Cycle 3: I1-EX  I2-ID  I3-IF
Cycle 4: I1-MEM  I2-EX  I3-ID  I4-IF
Cycle 5: I1-WB  I2-MEM  I3-EX  I4-ID  I5-IF
Cycle 6: I2-WB  I3-MEM  I4-EX  I5-ID
Cycle 7: I3-WB  I4-MEM  I5-EX
Cycle 8: I4-WB  I5-MEM
Cycle 9: I5-WB

===== PERFORMANCE =====
Number of Instructions = 5
Pipeline Stages = 5
Non-Pipelined Cycles = 25
Pipelined Cycles = 9
Speedup = 2.78
Throughput = 0.56 instructions/cycle
```

Result

The five-stage pipeline successfully executed the instruction sequence in **8 clock cycles**, compared with **20 clock cycles** in the non-pipelined system. The calculated speedup was **2.5**×.

Conclusion

Pipelining improves processor performance by allowing multiple instructions to be processed simultaneously in different stages. It significantly increases **instruction throughput** compared with sequential instruction execution.

Experiment 4 — Pipeline Hazards

Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact and implement stalling, data forwarding, and branch prediction to resolve the hazards.

Aim

To study **data, control, and structural hazards** in a pipelined processor and implement techniques such as **stalling, forwarding, and branch prediction** to improve pipeline performance.

Procedure

1. Create a five-stage pipeline.
2. Execute a sequence containing dependent instructions.
3. Identify data hazards.
4. Introduce a branch instruction to produce a control hazard.
5. Create resource conflicts to produce a structural hazard.
6. Observe the pipeline without hazard handling.
7. Implement pipeline stalling.
8. Implement data forwarding.
9. Implement branch prediction for control hazards.
10. Compare the number of cycles before and after hazard resolution.

Program / Simulation

```
#include <stdio.h>
int main()
{
   int normal_cycles = 9;
   int stall_cycles = 12;
   int forwarding_cycles = 10;
   int branch_prediction_cycles = 9;
   printf("===== PIPELINE HAZARD SIMULATION =====\n\n");
   printf("Instructions:\n");
   printf("I1: ADD R1, R2, R3\n");
   printf("I2: SUB R4, R1, R5\n");
   printf("I3: AND R6, R4, R7\n");
   printf("I4: BEQ R6, R0, TARGET\n");
   printf("I5: OR R8, R2, R3\n");
   printf("\n===== HAZARDS =====\n");
   printf("\n1. DATA HAZARD");
   printf("\nI2 depends on the result of I1.");
   printf("\n\n2. CONTROL HAZARD");
   printf("\nI4 is a branch instruction.");
   printf("\n\n3. STRUCTURAL HAZARD");
   printf("\nTwo instructions require the same hardware resource.");
   printf("\n\n===== HAZARD HANDLING =====");
```

```
    printf("\n\nWithout Hazard Handling:");
    printf("\nCycles = %d", normal_cycles);
    printf("\n\nWith Stalling:");
    printf("\nCycles = %d", stall_cycles);
    printf("\nStall Cycles = %d", stall_cycles - normal_cycles);
    printf("\n\nWith Data Forwarding:");
    printf("\nCycles = %d", forwarding_cycles);
    printf("\nStalls Reduced = %d",
           stall_cycles - forwarding_cycles);
    printf("\n\nWith Branch Prediction:");
    printf("\nCycles = %d",
           branch_prediction_cycles);
    printf("\n\n===== PERFORMANCE =====");
    printf("\nSpeedup using forwarding = %.2f",
           (float)stall_cycles /
           forwarding_cycles);
    printf("\nSpeedup with branch prediction = %.2f\n",
           (float)stall_cycles /
           branch_prediction_cycles);
    return 0;
}
```

Output

```
===== PIPELINE HAZARD SIMULATION =====

Instructions:
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: AND R6, R4, R7
I4: BEQ R6, R0, TARGET
I5: OR R8, R2, R3

===== HAZARDS =====

1. DATA HAZARD
I2 depends on the result of I1.

2. CONTROL HAZARD
I4 is a branch instruction.

3. STRUCTURAL HAZARD
Two instructions require the same hardware resource.

===== HAZARD HANDLING =====

Without Hazard Handling:
Cycles = 9

With Stalling:
Cycles = 12
Stall Cycles = 3

With Data Forwarding:
Cycles = 10
Stalls Reduced = 2

With Branch Prediction:
Cycles = 9

===== PERFORMANCE =====
Speedup using forwarding = 1.20
Speedup with branch prediction = 1.33
```

Result

The different types of pipeline hazards were successfully identified and simulated. Stalling resolved hazards by delaying instruction execution, while forwarding reduced unnecessary stalls. Branch prediction reduced the performance penalty caused by control hazards.

Conclusion

Pipeline hazards can significantly reduce the performance benefits of pipelining. **Data forwarding, hazard detection, stalling, and branch prediction** help maintain correct execution while reducing the number of lost clock cycles.

Experiment 5 — Superscalar, Out-of-Order and SMT Execution

Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to hyper-threading and simultaneous multithreading (SMT). Evaluate parallel instruction execution and compare instruction throughput and resource utilization.

Aim

To study **superscalar, out-of-order, speculative, and simultaneous multithreading (SMT)** execution and analyze their effects on instruction throughput and processor resource utilization.

Procedure

1. Design a processor model containing multiple execution units.
2. Allow more than one instruction to be issued in a clock cycle.
3. Implement multiple functional units such as:
    - ALU
    - Multiplication unit
    - Memory unit
4. Create instructions with different execution dependencies.
5. Execute them using in-order execution.
6. Execute the same instructions using out-of-order execution.
7. Introduce speculative execution for branch instructions.
8. Create two instruction streams representing two threads.
9. Execute them simultaneously using SMT.
10. Calculate IPC and instruction throughput.
11. Compare the performance of the different approaches.

Program / Simulation

```
#include <stdio.h>

int main()
{
   int instructions = 8;
   int scalarCycles = 8;
   int superscalarCycles = 4;
   int outOfOrderCycles = 3;
   int smtCycles = 4;
   float scalarIPC;
   float superscalarIPC;
   float outOfOrderIPC;
   float smtIPC;
   float superscalarSpeedup;
   float outOfOrderSpeedup;
   float smtSpeedup;
   printf("========================================\n");
```

```
    printf(" SUPERSCALAR AND SMT SIMULATION\n");
    printf("=========================================\n");
    printf("\nInstruction Sequence:\n");
    printf("I1: ADD R1, R2, R3\n");
    printf("I2: MUL R4, R5, R6\n");
    printf("I3: SUB R7, R1, R8\n");
    printf("I4: AND R9, R10, R11\n");
    printf("I5: OR  R12, R7, R13\n");
    printf("I6: ADD R14, R15, R16\n");
    printf("I7: MUL R17, R18, R19\n");
    printf("I8: SUB R20, R21, R22\n");
    printf("\n=========================================\n");
    printf("            EXECUTION\n");
    printf("=========================================\n");
    printf("\n1. SCALAR PROCESSOR\n");
    printf("Instructions = %d\n", instructions);
    printf("Cycles = %d\n", scalarCycles);
    printf("\n2. 2-WIDE SUPERSCALAR\n");
    printf("Instructions can be issued in pairs.\n");
    printf("Cycles = %d\n", superscalarCycles);
    printf("\n3. OUT-OF-ORDER EXECUTION\n");
    printf("Independent instructions can execute early.\n");
    printf("Cycles = %d\n", outOfOrderCycles);
    printf("\n4. SMT EXECUTION\n");
    printf("Instructions from two threads execute concurrently.\n");
    printf("Cycles = %d\n", smtCycles);
    scalarIPC =(float)instructions /scalarCycles;
    superscalarIPC =(float)instructions /superscalarCycles;
    outOfOrderIPC =(float)instructions /outOfOrderCycles;
    smtIPC =float)instructions /smtCycles;
    superscalarSpeedup =(float)scalarCycles /superscalarCycles;
    outOfOrderSpeedup =(float)scalarCycles /outOfOrderCycles;
    smtSpeedup =(float)scalarCycles /smtCycles;
    printf("\n=========================================\n");
    printf("       PERFORMANCE COMPARISON\n");
    printf("=========================================\n");
    printf("\nArchitecture\t\tCycles\tIPC\n");
    printf("Scalar\t\t\t%d\t%.2f\n",
           scalarCycles,
           scalarIPC);
    printf("Superscalar\t\t%d\t%.2f\n",
           superscalarCycles,
           superscalarIPC);
    printf("Out-of-Order\t\t%d\t%.2f\n",
           outOfOrderCycles,
           outOfOrderIPC);
    printf("SMT\t\t\t%d\t%.2f\n",
           smtCycles,
           smtIPC);
    printf("\n=========================================\n");
    printf("            SPEEDUP\n");
    printf("=========================================\n");
```

```
    printf("\nSuperscalar Speedup = %.2f\n",
           superscalarSpeedup);
    printf("Out-of-Order Speedup = %.2f\n",
           outOfOrderSpeedup);
    printf("SMT Speedup = %.2f\n",
           smtSpeedup);
    printf("\n========================================\n");
    printf("Simulation completed successfully.\n");
    printf("========================================\n");
    return 0;
}
```

Output

```
========================================
 SUPERSCALAR AND SMT SIMULATION
========================================

Instruction Sequence:
I1: ADD R1, R2, R3
I2: MUL R4, R5, R6
I3: SUB R7, R1, R8
I4: AND R9, R10, R11
I5: OR  R12, R7, R13
I6: ADD R14, R15, R16
I7: MUL R17, R18, R19
I8: SUB R20, R21, R22

========================================
             EXECUTION
========================================

1. SCALAR PROCESSOR
Instructions = 8
Cycles = 8

2. 2-WIDE SUPERSCALAR
Instructions can be issued in pairs.
Cycles = 4

3. OUT-OF-ORDER EXECUTION
Independent instructions can execute early.
Cycles = 3

4. SMT EXECUTION
Instructions from two threads execute concurrently.
Cycles = 4

========================================
        PERFORMANCE COMPARISON
========================================

Architecture            Cycles  IPC
Scalar                  8       1.00
Superscalar             4       2.00
Out-of-Order            3       2.67
SMT                     4       2.00

========================================
             SPEEDUP
========================================

Superscalar Speedup = 2.00
Out-of-Order Speedup = 2.67
SMT Speedup = 2.00

========================================
Simulation completed successfully.
========================================
```

Result

Superscalar execution successfully executed multiple independent instructions in parallel. Out-of-order execution improved the utilization of available execution units by allowing independent instructions to proceed before stalled instructions. SMT further improved resource utilization by allowing instructions from multiple threads to execute concurrently.

Conclusion

Superscalar and out-of-order execution improve processor performance by exploiting **instruction-level parallelism**. Speculative execution reduces the impact of branches, while **SMT/hyper-threading** improves utilization of processor resources by executing instructions from multiple threads concurrently.